MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

IncidentFlow: A Domain-Specific Language for Cybersecurity Incident Reporting

Project report

Mentor: prof., Brânzan Leon
Students: Kushnirenko Ekaterina, FAF-243
Felicia Ojog, FAF-243
Botezatu Gabriela, FAF-243
Șoimu Ionuț, FAF-243

Chișinău, 2025

Abstract

This paper presents IncidentFlow, a domain-specific language (DSL) designed to improve cybersecurity incident reporting by reducing manual authoring effort and structural inconsistency. In current practice, incident reports are often produced through static templates and post-hoc editing workflows, where responders must repeatedly verify required sections, formatting constraints, and compliance-oriented evidence. As analyzed in Chapter 1, this process introduces procedural drift, weakens coordination visibility, and increases auditability risk when documentation is assembled under operational pressure.

The proposed solution models incident response and reporting within a single formal specification. Instead of reconstructing reports from fragmented notes, the DSL encodes triggers, phases, control flow, verification actions, severity transitions, and reporting events as first-class language constructs. This representation enables reporting requirements to be enforced by syntax and structure, thereby reducing repetitive requirement-checking by end users.

Implementation is based on an ANTLR-driven language pipeline, including a formally defined grammar, dedicated lexer rules, and parser logic with deterministic error handling, as detailed in Chapter 2. The resulting architecture provides a validated parse model that can support downstream automation of report generation and consistency checks.

The main contribution is a practical, extensible foundation for process-aware incident documentation that improves efficiency, reduces human error, and strengthens traceable reporting quality in high-pressure security operations.

Keywords: Domain-Specific Languages, Cybersecurity Incident Response, Incident Reporting Automation, ANTLR, Parser Design, Compliance Engineering, Human Error Reduction.

Content

Introduction **4**

Domain Analysis and Problem Framing **5**

1.1 Problem Description and Problem Analysis 5

1.1.1 Domain Overview: Cybersecurity Incident Response 5

1.2 Description of the DSL 10

Language Architecture and Implementation **11**

2.1 Grammar Structure 11

2.2 Lexer Rules 14

2.3 Parser Logic 16

2.4 Derivation Tree Analysis 17

Conclusions **20**

Appendix **21**

Bibliography **38**

Introduction

Cybersecurity incidents are handled under pressure, but incident reports are still often produced with manual, template-driven work. Analysts and incident leads must investigate technical evidence, coordinate decisions, and document actions in parallel, while also making sure the final report follows predefined structures and mandatory fields. In practice, this creates a recurring burden: teams repeatedly pause to check whether they have respected all template requirements, formatting rules, and reporting constraints. The process becomes slow, error-prone, and hard to sustain during active incidents.

This project focuses on improving that situation through usability and automation. The core idea is that reporting should be generated from the incident-handling workflow itself, not written from scratch after the response ends. When key actions, timestamps, decisions, and ownership are captured in structured form during execution, the system can automatically assemble most of the report content. This reduces duplicate work, keeps information consistent across sections, and allows teams to spend less time verifying checklist items in static templates.

An important objective is to reduce the constant need to manually validate compliance with report templates. Instead of repeatedly checking if each required section exists and whether every mandatory detail has been included, responders should receive built-in guidance that enforces structure as they work. In this approach, template requirements are embedded into the flow, so completeness and consistency become default outcomes rather than late-stage corrections. The benefit is both operational and organizational: faster reporting cycles, fewer omissions, and clearer artifacts for management and audit use.

The project also emphasizes ease of use for different roles involved in incident response. A usable system should make the next step explicit, minimize unnecessary data entry, and preserve human judgment where it matters most. Automation is treated as support, not replacement: it handles repetitive formatting and report composition tasks, while analysts remain responsible for interpretation, prioritization, and decision-making. This balance is essential for producing reports that are both technically reliable and understandable to non-technical stakeholders.

The rest of the report follows this direction. The next chapter analyzes the incident response domain and identifies the procedural and reporting gaps that motivate the proposed solution. It then introduces the initial concept for executable guidance and automated report generation. The final chapter summarizes the current results and outlines the next steps. Overall, the project aims to make incident reporting accurate, timely, and significantly easier by integrating template compliance and report generation directly into daily response workflows.

Domain Analysis and Problem Framing

1.1 Problem Description and Problem Analysis

The origin of the project is the problem itself. Naturally, the project team should continue the work on the project by building their way around the problem description. A good description of the problem forms the foundation of the other aspects and allows the team to make progress. Any problem has its impact in the scope of one or multiple domains. It is important to identify this scope so that the problem could be analyzed and its impact over the domain assessed. Having that in place, the project team can come up with a concept for a solution to the analyzed problem. [1]

1.1.1 Domain Overview: Cybersecurity Incident Response

Cybersecurity incident response (IR) is one of the most operationally demanding disciplines within the broader field of information security. As organizations increasingly depend on digital infrastructure, the frequency, sophistication, and financial impact of cyberattacks have grown substantially. According to IBM's *Cost of a Data Breach Report 2023*, the global average cost of a data breach reached $4.45 million USD, a record high at the time of publication [2]. This environment places enormous pressure on security teams to detect, analyze, and recover from incidents in a structured, repeatable, and auditable manner. The domain of incident response sits at the intersection of technical operations, organizational governance, and regulatory compliance, making it inherently multidisciplinary and complex.

What Is Incident Response?

Incident response is the organized approach to addressing and managing the aftermath of a security breach or cyberattack. The primary goals are to limit damage, reduce recovery time and costs, preserve forensic evidence, and prevent future recurrences [3]. A *security incident* is formally defined by NIST as "a violation or imminent threat of violation of computer security policies, acceptable use policies, or standard security practices" [3]. This definition encompasses a wide range of events, from malware infections and ransomware deployments to insider threats, data exfiltration, and distributed denial-of-service (DDoS) attacks.

The discipline is governed by several internationally recognized frameworks and standards. The most widely adopted in North America is NIST Special Publication 800-61 *Computer Security Incident Handling Guide*, published and maintained by the National Institute of Standards and Technology [3]. In Europe and internationally, ISO/IEC 27035 *Information Security Incident Management* provides a comparable structured methodology [4]. The SANS Institute additionally publishes its own IR methodology and maintains the widely referenced *Incident Handler's Handbook*, which is extensively used for practitioner training [5]. While these frameworks differ in scope and terminology, they share a common structural

philosophy: incident response must be lifecycle-based, documented, and continuously improved.

The Incident Response Lifecycle (NIST SP 800-61)

NIST SP 800-61 defines the incident response lifecycle as consisting of four primary phases, each with distinct objectives and activities [3]. These phases are not always strictly sequential; in practice, responders may cycle between phases or execute certain activities in parallel depending on the nature and scope of the incident.

Phase 1: Preparation. The preparation phase encompasses all activities that take place before an incident occurs. This includes establishing incident response policies and procedures, defining team roles and responsibilities, procuring and configuring response tooling, creating and maintaining *playbooks* (step-by-step procedural guides for specific incident types), setting up communication and escalation plans, and conducting tabletop exercises and simulations. NIST emphasizes that preparation is arguably the most critical phase because the quality of preparation directly determines the effectiveness of all subsequent phases [3]. Organizations that lack mature preparation capabilities tend to exhibit significantly longer mean time to detect (MTTD) and mean time to respond (MTTR) metrics. According to IBM's 2023 report, organizations with a fully deployed IR team and regularly tested IR plans saved an average of $1.49 million USD compared to those without such capabilities [2].

Phase 2: Detection and Analysis. The detection and analysis phase involves identifying potential security events and determining whether they constitute actual incidents. Detection sources include Security Information and Event Management (SIEM) platforms, Intrusion Detection and Prevention Systems (IDS/IPS), endpoint detection and response (EDR) tools, network traffic analysis, user reports, and third-party threat intelligence feeds [3]. The analytical challenge at this phase is substantial: Security Operations Center (SOC) analysts are routinely exposed to hundreds or thousands of alerts per shift, the overwhelming majority of which are false positives. A 2022 study by Ponemon Institute found that SOC analysts spend approximately 27% of their time investigating alerts that turn out to be non-incidents, contributing to analyst burnout and reduced organizational security posture [6]. Effective detection requires not only robust tooling but well-defined triage procedures and severity classification criteria, typically organized into tiered priority levels (P1 through P4 or similar) that guide how quickly and intensively analysts must respond.

Phase 3: Containment, Eradication, and Recovery. Once an incident is confirmed and analyzed, the response team must act to limit its spread (containment), remove the threat actor and all malicious artifacts from affected systems (eradication), and restore normal operations from a known-good state (recovery) [3]. Containment strategies range from short-term isolation of individual hosts to full network segmentation, and the choice depends heavily on the incident type and the organization's operational tolerance for disruption. Eradication may involve reimaging systems, revoking compromised credentials, patching exploited vulnerabilities, and removing persistence mechanisms. Recovery involves restoring services from clean backups,

verifying system integrity, and gradually returning systems to production under close monitoring. A critical challenge throughout this phase is the tension between speed and thoroughness: moving too quickly risks missing residual attacker footholds, while excessive caution prolongs business disruption [5].

Phase 4: Post-Incident Activity. The final phase encompasses all activities that occur after the incident has been resolved. These include conducting a formal lessons-learned review, updating IR playbooks and detection rules based on findings, producing regulatory and management reports, preserving forensic evidence in accordance with legal requirements, and filing breach notifications where mandated by law [3]. This phase is frequently underinvested in practice: a 2021 survey by ISACA found that fewer than 40% of organizations conducted formal post-incident reviews after every security incident [7]. The consequence is a failure to incorporate operational knowledge back into preparation materials, leading to repeated errors in future incidents.

The Role of Playbooks in Incident Response

Incident response playbooks are structured, procedural documents that prescribe the exact steps an analyst must follow when handling a specific category of incident, such as ransomware, phishing, account compromise, or data exfiltration. Playbooks operationalize the high-level guidance of frameworks like NIST SP 800-61 into actionable, role-specific instructions. They typically include decision trees, escalation criteria, specific system commands, communication templates, and approval gates [5].

In practice, playbooks are most commonly stored and distributed as static documents — PDF files, Word documents, or wiki pages — that analysts must read and manually follow under time pressure. This creates a significant operational risk: research published in the *Journal of Cybersecurity* has highlighted that cognitive load during high-stress incident response scenarios leads to procedural non-compliance, step skipping, and documentation gaps [8]. Furthermore, static playbooks are difficult to keep current; as threat landscapes evolve and organizational environments change, outdated procedures can mislead responders rather than guide them.

The concept of *executable playbooks* or *security orchestration, automation, and response* (SOAR) platforms has emerged as a technical response to these challenges. SOAR platforms such as Splunk SOAR (formerly Phantom), Palo Alto XSOAR, and IBM Resilient allow organizations to encode playbook logic into automated workflows that execute actions, track state, and enforce procedural compliance programmatically [9]. However, enterprise SOAR platforms are expensive, complex to deploy, and typically accessible only to large organizations with mature security operations, leaving small and medium-sized enterprises (SMEs) without viable alternatives to static document-based playbooks.

Regulatory and Compliance Dimensions

Incident response does not operate in a legal vacuum. Organizations operating in or serving customers in regulated environments are subject to mandatory breach notification and incident documentation

requirements under a growing body of legislation and regulation.

The European Union's *General Data Protection Regulation* (GDPR, Regulation 2016/679) requires organizations to notify the relevant supervisory authority within 72 hours of becoming aware of a personal data breach, and to notify affected individuals without undue delay where the breach is likely to result in high risk [10]. The *Network and Information Security Directive* (NIS2 Directive, 2022/2555/EU), which came into force in October 2024, extends mandatory incident reporting obligations to a significantly broader range of sectors deemed critical infrastructure across EU member states, with substantial financial penalties for non-compliance [11]. In the United States, sector-specific regulations such as the *Sarbanes-Oxley Act* (SOX) impose obligations on publicly traded companies to maintain accurate records of material cybersecurity events affecting financial reporting systems [12]. The *Health Insurance Portability and Accountability Act* (HIPAA) similarly mandates breach notification and documentation requirements in the healthcare sector [13].

A common thread across all of these regulatory frameworks is the requirement for *demonstrable, auditable evidence* that incidents were handled according to defined procedures. This creates a documentation burden that, under the pressure of an active incident, is frequently deprioritized in favor of immediate technical response actions. The resulting gap between what organizations claim their IR procedures are and what actually occurred during an incident is a recognized compliance risk, particularly during post-breach regulatory audits [7].

Stakeholders in Incident Response Operations

Effective incident response requires coordinated action across multiple organizational roles, each with distinct responsibilities, information needs, and operational pain points. Understanding the stakeholder landscape is essential to identifying where operational gaps and inefficiencies arise within the IR domain.

The diversity of stakeholder needs highlights a structural challenge within the IR domain: different roles require different views and interfaces into the same underlying incident data, yet most organizations rely on a single shared artifact — the static playbook document — to coordinate all of these perspectives simultaneously.

Identified Gaps and Problem Areas in the Domain

Drawing on the preceding analysis of the IR lifecycle, playbook practices, regulatory requirements, and stakeholder needs, several recurring problem areas can be identified that represent systemic weaknesses within the cybersecurity incident response domain.

The first and most significant problem is the *procedural compliance gap* introduced by static, document-based playbooks. When analysts must manually follow PDF or wiki-based procedures during high-pressure incidents, cognitive load and time constraints routinely lead to steps being skipped, executed out of order, or left undocumented [8]. This is not a failure of individual competence but a structural

Table 1.1.1 - Key Stakeholders in Cybersecurity Incident Response

Role	Responsibility	Operational Pain Points
SOC Analyst (Tier 1/2)	First responder; triages alerts, executes playbook procedures, collects forensic evidence, and escalates as appropriate	Reads static PDF playbooks under acute time pressure; prone to step skipping and incomplete documentation; alert fatigue from high false-positive volumes
SOC Lead / Incident Commander	Supervises and coordinates the response effort; makes real-time escalation and containment decisions; manages inter-team communication	Lacks real-time visibility into step-level response progress; cannot identify bottlenecks until after-action review; dependent on verbal status updates from analysts
CISO / Security Director	Strategic oversight of the IR program; accountable for regulatory compliance; responsible for executive and board-level reporting	Cannot validate the procedural correctness of playbooks before incidents occur; cannot assess coverage gaps across incident types; relies on after-action reports that may omit critical details
Compliance and Audit	Ensures the organization adheres to applicable regulatory frameworks (GDPR, NIS2, SOX, HIPAA); prepares documentation for external auditors	Cannot produce reliable evidence that playbooks were actually followed step-by-step; timeline reconstruction from disparate logs is labor-intensive and error-prone
IT Operations	Maintains infrastructure; performs system isolation, backup restoration, and recovery verification tasks during and after incidents	Receives recovery instructions via informal channels (verbal, email, chat) during crisis conditions; lacks structured, verified recovery procedures
Legal Counsel	Advises on breach notification obligations; assesses litigation exposure; liaises with regulatory bodies	Requires precise timeline of events and response actions for regulatory filings and potential litigation; this data is rarely captured in a structured, defensible format

consequence of using passive documentation artifacts to govern active, high-stakes operational processes.

The second problem is the *visibility and coordination deficit* experienced by SOC leads and CISOs during active incidents. Without real-time insight into which procedural steps have been completed, by whom, and with what outcome, senior stakeholders cannot make informed escalation decisions or identify response bottlenecks until the incident is resolved and an after-action report is compiled. This lag between operational reality and management awareness is a well-documented contributor to extended incident dwell

times [2].

The third problem is the *auditability gap* that emerges at the intersection of operational IR and regulatory compliance. Compliance frameworks require organizations to demonstrate that documented procedures were followed; however, the informal, ad-hoc documentation practices common during live incidents rarely produce the structured, timestamped, analyst-attributed evidence necessary to satisfy regulatory audit requirements [7].

The fourth problem is the *playbook quality assurance deficit*. Because playbooks are stored as static documents, there is no systematic mechanism for detecting logical errors, missing steps, undefined variables, or coverage gaps before an incident occurs. Quality control is largely informal and depends on periodic manual review, which is inconsistently performed [5].

These four problem areas collectively define the operational and organizational context within which the present project is situated.

1.2 Description of the DSL

The IncidentFlow DSL is proposed as a representational solution to this problem. Instead of treating reporting as a final narrative that is manually composed from scattered operational artifacts, the DSL encodes incident logic directly in a structured, machine-parseable form. The language unifies operational flow and reporting semantics so that required report content emerges from validated process execution.

At the model level, the DSL separates reusable context from incident-specific behavior through configuration blocks, imports, and playbook declarations. Within playbooks, the language supports triggers, named phases, conditional branches, iteration, timeout-aware waiting, verification handlers, severity transitions, logging, and explicit control transfer. This design captures the essential response semantics needed for both execution guidance and report reconstruction.

From a usability perspective, the key value is reduction of repetitive manual requirement-checking. If required reporting elements are represented as first-class language constructs (for example, explicit phase boundaries, severity updates, and closure actions), users do not need to repeatedly inspect external templates to verify structural completeness. Completeness becomes a property of the specification and validation pipeline, not a late-stage manual proofreading task.

From an architectural perspective, the DSL provides deterministic syntax and a stable parse-tree representation that can support downstream automation. These properties enable future capabilities such as generated report templates, policy conformance validation, timeline reconstruction, and evidence packaging. In this sense, the DSL is both a domain notation and an integration surface between operations and governance.

Language Architecture and Implementation

2.1 Grammar Structure

The IncidentFlow grammar is designed as a domain-oriented formal specification that models how incident procedures are described, validated, and prepared for automated report generation. At the top level, the language is intentionally constrained: a valid program can contain configuration blocks, import declarations, and one or more playbook declarations. This creates a predictable document structure and prevents partially defined incident procedures from being accepted as complete specifications.

```
program
    : config_block* import_decl* playbook_decl+ EOF
    ;

import_decl
    : IMPORT STRING (AS IDENTIFIER)?
    ;

config_block
    : CONFIG config_name COLON config_entry+
    ;
```

From an architectural perspective, this top-level design enforces a useful separation of concerns. Configuration captures stable, reusable organizational metadata, while playbooks capture incident-specific behavior. The report model then becomes derivable from explicit execution constructs instead of informal prose. This is the first step toward reducing manual requirement-checking and enforcing template completeness through syntax.

The grammar models incident lifecycle behavior with explicit declarations for triggers, phases, and report blocks. A playbook is therefore not a flat script but a structured operational narrative. Triggers define activation conditions, phases encode controlled progression, and report sections capture closure actions.

```
playbook_decl
    : PLAYBOOK IDENTIFIER trigger_decl phase_decl+ report_decl?
    ;

trigger_decl
    : TRIGGER COLON expr
    ;
```

```

phase_decl
    : PHASE IDENTIFIER COLON statement+
    ;

report_decl
    : REPORT COLON statement+
    ;
```

This structure directly supports reporting automation: each phase contributes traceable execution events, and the report declaration provides explicit closing semantics. Instead of asking users to re-check external templates for required sections, required reporting elements are represented in the grammar itself.

The statement system is expressive enough for realistic incident handling while remaining intentionally limited to domain-relevant constructs. The grammar includes execution actions, conditional paths, iteration, waiting with timeout handling, verification with failure handling, logging, control transfer, assignment, parallel execution intent, and severity control.

```
statement
    : do_stmt
    | if_stmt
    | for_stmt
    | wait_stmt
    | verify_stmt
    | log_stmt
    | goto_stmt
    | assign_stmt
    | parallel_block
    | severity_stmt
    ;

wait_stmt
    : WAIT IDENTIFIER FROM expr TIMEOUT DURATION timeout_handler?
    ;

verify_stmt
    : VERIFY expr fail_handler?
    ;
```

Expression rules are organized by precedence to avoid ambiguity and to keep condition semantics deterministic. This is essential for reproducibility and future static analysis.

```
expr
    : or_expr
    ;

or_expr
    : and_expr (OR and_expr)*
    ;

and_expr
    : not_expr (AND not_expr)*
    ;

not_expr
    : NOT not_expr
    | comparison
    ;

comparison
    : primary (comp_op primary)?
    ;
```

The literal and access model supports practical policy authoring. In addition to primitive literals, the language supports list literals, member access, and chain literals, which are useful for escalation sequences and organizational routing metadata.

```
config_value
    : STRING
    | INT
    | severity_lit
    | bool_lit
    | list_lit
    | chain_lit
    ;

```

```
list_lit
    : LBRACKET config_value (COMMA config_value)* RBRACKET
    ;

chain_lit
    : IDENTIFIER (ARROW IDENTIFIER)+
    ;
```

Finally, the grammar balances strictness and extensibility. Strictness is provided by explicit keyword-driven forms and mandatory structural elements. Extensibility is provided by modular rules that can evolve without changing core parse entry points. This combination makes the grammar suitable for iterative DSL growth while maintaining compatibility for automated report generation pipelines.

2.2 Lexer Rules

The lexer defines the lexical contract of the language and transforms source text into domain-aware tokens used by the parser. In IncidentFlow, lexical design is intentionally explicit: operational terms are reserved as keywords so the syntax remains self-documenting and less error-prone.

```
PLAYBOOK    : 'PLAYBOOK' ;
CONFIG      : 'CONFIG' ;
TRIGGER     : 'TRIGGER' ;
PHASE       : 'PHASE' ;
DO          : 'DO' ;
IF          : 'IF' ;
THEN        : 'THEN' ;
ELSE        : 'ELSE' ;
WAIT        : 'WAIT' ;
TIMEOUT     : 'TIMEOUT' ;
ON_TIMEOUT  : 'ON_TIMEOUT' ;
VERIFY      : 'VERIFY' ;
ON_FAIL     : 'ON_FAIL' ;
REPORT      : 'REPORT' ;
SEVERITY    : 'SEVERITY' ;
```

Domain literals are captured with dedicated token rules. The objective is to validate typing constraints as early as possible and avoid postponing basic correctness checks to semantic phases.

```
IDENTIFIER  : [a-zA-Z_][a-zA-Z0-9_]* ;
STRING      : '"' (~["\\\r\n] | '\\' .)* '"' ;
```

```
DURATION    : [0-9]+ ('sec' | 'min' | 'hr') ;
INT         : [0-9]+ ;

TRUE        : 'TRUE' ;
FALSE       : 'FALSE' ;
LOW         : 'LOW' ;
MEDIUM      : 'MEDIUM' ;
HIGH        : 'HIGH' ;
CRITICAL    : 'CRITICAL' ;
```

Operator rules are declared with careful ordering to preserve longest-match behavior and avoid tokenization conflicts. This is particularly important for comparison operators where prefix overlap exists.

```
EQ          : '==' ;
NEQ         : '!=' ;
GTE         : '>=' ;
LTE         : '<=' ;
GT          : '>' ;
LT          : '<' ;

ASSIGN      : '=' ;
ARROW       : '->' ;
COLON       : ':' ;
COMMA       : ',' ;
DOT         : '.' ;
LPAREN      : '(' ;
RPAREN      : ')' ;
LBRACKET    : '[' ;
RBRACKET    : ']' ;
```

Lexical sanitation is handled through explicit skip rules for comments, whitespace, and BOM markers. This keeps token streams stable across operating systems and editor configurations.

```
COMMENT       : '#' ~[\r\n]* -> skip ;
BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
WHITESPACE    : [ \t\r\n]+ -> skip ;
BOM           : '\uFEFF' -> skip ;
```

The lexer therefore contributes directly to usability goals: authors can write concise, readable playbooks using domain terms, while the implementation enforces lexical consistency automatically.

2.3 Parser Logic

Parser logic is implemented through generated parser classes and a dedicated runtime driver. The runtime flow follows a standard but robust sequence: source loading, character stream creation, lexical analysis, token stream construction, parser invocation, and parse-tree materialization through the program entry rule.

```
CharStream input = CharStreams.fromString(source);
IncidentFlowLexer lexer = new IncidentFlowLexer(input);
CommonTokenStream tokens = new CommonTokenStream(lexer);
IncidentFlowParser parser = new IncidentFlowParser(tokens);
ParseTree tree = parser.program();
```

A key design decision is centralized syntax error accounting through a custom listener attached to both lexer and parser. This provides a uniform diagnostic channel and supports fail-fast termination when invalid constructs are present.

```
ErrorCounter errors = new ErrorCounter();
lexer.removeErrorListeners();
lexer.addErrorListener(errors);
parser.removeErrorListeners();
parser.addErrorListener(errors);

if (errors.getCount() > 0) {
    System.err.println("Parsing finished with " + errors.getCount() + " error(s).");
    System.exit(2);
}
```

The error listener implementation is lightweight and deterministic, which is appropriate for build pipelines where syntax validity must be an objective gate.

```
private static class ErrorCounter extends BaseErrorListener {
    private int count = 0;

    @Override
    public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
                            int line, int charPositionInLine,
                            String msg, RecognitionException e) {
```

```
        System.err.printf("line %d:%d %s%n", line, charPositionInLine, msg);
        count++;
    }

    int getCount() { return count; }
}
```

From a parser architecture viewpoint, three properties are relevant for the DSL objective. First, structural validation is guaranteed before execution or report generation begins. Second, parse trees preserve full syntactic context, enabling deterministic extraction of phase transitions, severity changes, and report actions. Third, parser output can be transformed into intermediate models for downstream generators that produce human-readable reports without requiring users to manually satisfy template constraints.

The generation and execution workflow is automated through a build script that regenerates parser artifacts, compiles runtime classes, and validates a representative DSL sample. This allows grammar evolution to be tested continuously.

```
java -jar "${ANTLR_JAR}" \
    -o gen \
    -package incidentflow \
    -visitor \
    -listener \
    IncidentFlow.g4

javac -cp "${ANTLR_JAR}" -d bin $(find gen -name '*.java') Main.java
java -cp "bin:${ANTLR_JAR}" incidentflow.Main sample.iflow
```

In summary, parser logic in IncidentFlow is intentionally conservative and reliability-oriented. The implementation favors explicit syntax, deterministic error reporting, and reproducible parse outcomes. These properties are necessary for a DSL whose primary architectural promise is trustworthy automation of incident reporting.

2.4 Derivation Tree Analysis

The derivation tree is the most direct structural representation of how an IncidentFlow specification is recognized by the grammar. Whereas token streams capture lexical units and parser rules define admissible forms, the derivation tree materializes the full hierarchical decomposition from the start symbol to terminal constructs. This makes the tree an important artifact for grammar validation, semantic mapping, and quality assurance in language evolution.

For readability and layout continuity, the full derivation tree visual is provided in Appendix A.4.

At the root level, the tree is anchored in `program`, which expands into optional configuration and import regions, followed by one or more `playbook_decl` nodes. This root structure is architecturally significant because it enforces a canonical specification order and prevents malformed documents where operational logic exists without declarative context.

The next layer captures playbook identity and activation logic. A `playbook_decl` node expands into the playbook name, a mandatory `trigger_decl`, one or more `phase_decl` blocks, and an optional `report_decl`. In the tree, this pattern makes the lifecycle skeleton explicit: activation condition, staged response behavior, and closure/reporting behavior. The tree therefore encodes not only syntax, but a normalized operational narrative.

Within each phase, statement nodes form deeply nested subtrees that correspond to control-flow semantics. Conditional branches generate `if_stmt` subtrees with explicit `THEN` and optional `ELSE` branches; loops generate `for_stmt` branches over iterable expressions; timeout and failure handling produce nested handler branches under `wait_stmt` and `verify_stmt`. This nesting is essential for downstream transformations, since it preserves execution dependencies and fallback logic that are often lost in flat, manually written reports.

Expression subtrees reveal another critical property: precedence-aware normalization. Nodes for `or_expr`, `and_expr`, `not_expr`, and `comparison` are arranged according to grammar precedence, ensuring that complex trigger and verification predicates are represented unambiguously. For report automation, this is important because condition rationale can be reconstructed from tree structure rather than inferred from free-form text.

The tree also provides a bridge between procedural and reporting semantics. Subtrees containing `LOG`, `SEVERITY`, `VERIFY`, and closure actions in `report_decl` can be mapped to report sections such as timeline events, severity evolution, validation outcomes, and closure summary. In other words, the derivation tree provides a deterministic intermediate representation from which reporting artifacts can be generated with consistent structure.

To illustrate structural depth, the following simplified textual projection shows the core hierarchy observed in a typical sample parse:

```
program
|- config_block*
|- import_decl*
`- playbook_decl+
   |- PLAYBOOK IDENTIFIER
   |- trigger_decl
```

```
   |  `- expr
   |- phase_decl+
   |  |- PHASE IDENTIFIER
   |  `- statement+
   |     |- if_stmt
   |     |- wait_stmt
   |     |- verify_stmt
   |     |- do_stmt
   |     `- severity_stmt
   `- report_decl?
      `- statement+
```

From an engineering perspective, this tree-centric view supports three advanced activities. First, it enables regression testing of grammar changes by comparing expected and actual subtree shapes for representative inputs. Second, it allows semantic passes to consume stable node patterns rather than brittle textual heuristics. Third, it improves diagnosability: when parse errors occur, developers can localize failures by identifying which subtree expansion failed and at what structural depth.

Overall, the derivation tree is not a debugging convenience but a core architectural artifact in the IncidentFlow pipeline. It demonstrates that the language can express incident procedures as formally constrained structures, and it provides the foundation for reliable translation from authored playbooks to automation-ready reporting representations.

Conclusions

This report analyzed the incident reporting domain and demonstrated why manual, template-driven workflows remain a significant operational bottleneck. The analysis showed that repeated requirement-checking, duplicated data entry, and late-stage formatting validation reduce response efficiency and negatively affect report consistency. These issues are structural and cannot be reliably solved through stricter discipline alone.

The IncidentFlow DSL addresses this gap by moving from document-centric reporting to process-centric specification. By encoding triggers, phases, verification points, severity transitions, and report actions in a formal grammar, the approach enables consistent structure and predictable execution semantics. In practical terms, users spend less effort validating template compliance because required reporting elements are captured through language constructs rather than after-the-fact manual editing.

The ANTLR-based implementation confirms technical feasibility. The grammar defines a clear syntactic contract, the lexer provides domain-aligned tokenization, and the parser pipeline offers deterministic validation with explicit error handling. Together, these components establish a foundation for automated report generation, procedural quality control, and future integration with operational security tooling.

Overall, the DSL contributes both usability and governance value. It improves the end-user experience by reducing cognitive overhead and repetitive formatting tasks, while also improving organizational outcomes through more complete, auditable, and reproducible incident reports. The project therefore demonstrates that a domain-specific language can serve as a practical architectural mechanism for turning incident documentation from a manual burden into a controlled and automatable process.

Appendix

Project repository: `https://github.com/nnorian/IncidentFlow_dsl/`

A.1 Complete Grammar Specification

```
grammar IncidentFlow;

program
    : config_block* import_decl* playbook_decl+ EOF
    ;

import_decl
    : IMPORT STRING (AS IDENTIFIER)?
    ;

config_block
    : CONFIG config_name COLON config_entry+
    ;

config_name
    : IDENTIFIER
    ;

config_entry
    : IDENTIFIER COLON config_value
    ;

config_value
    : STRING
    | INT
    | severity_lit
    | bool_lit
    | list_lit
    | chain_lit
    ;

```

```
list_lit
    : LBRACKET config_value (COMMA config_value)* RBRACKET
    ;

chain_lit
    : IDENTIFIER (ARROW IDENTIFIER)+
    ;

playbook_decl
    : PLAYBOOK IDENTIFIER trigger_decl phase_decl+ report_decl?
    ;

trigger_decl
    : TRIGGER COLON expr
    ;

phase_decl
    : PHASE IDENTIFIER COLON statement+
    ;

report_decl
    : REPORT COLON statement+
    ;

statement
    : do_stmt
    | if_stmt
    | for_stmt
    | wait_stmt
    | verify_stmt
    | log_stmt
    | goto_stmt
    | assign_stmt
    | parallel_block
    | severity_stmt
    ;
```

```
do_stmt
    : DO function_call
    ;

if_stmt
    : IF expr THEN COLON statement+ (ELSE COLON statement+)?
    ;

for_stmt
    : FOR EACH IDENTIFIER IN expr COLON statement+
    ;

wait_stmt
    : WAIT IDENTIFIER FROM expr TIMEOUT DURATION timeout_handler?
    ;

timeout_handler
    : ON_TIMEOUT COLON statement+
    ;

verify_stmt
    : VERIFY expr fail_handler?
    ;

fail_handler
    : ON_FAIL COLON statement+
    ;

log_stmt
    : LOG expr
    ;

goto_stmt
    : GOTO IDENTIFIER
    ;
```

```
assign_stmt
    : IDENTIFIER ASSIGN expr
    ;

parallel_block
    : PARALLEL COLON statement+
    ;

severity_stmt
    : SEVERITY severity_lit
    ;

expr
    : or_expr
    ;

or_expr
    : and_expr (OR and_expr)*
    ;

and_expr
    : not_expr (AND not_expr)*
    ;

not_expr
    : NOT not_expr
    | comparison
    ;

comparison
    : primary (comp_op primary)?
    ;

comp_op
    : EQ
```

```
    | NEQ
    | GTE
    | LTE
    | GT
    | LT
    ;

primary
    : function_call
    | member_access
    | IDENTIFIER
    | literal
    | LPAREN expr RPAREN
    ;

member_access
    : IDENTIFIER (DOT IDENTIFIER)+
    ;

function_call
    : IDENTIFIER LPAREN argument_list? RPAREN
    ;

argument_list
    : argument (COMMA argument)*
    ;

argument
    : IDENTIFIER COLON expr
    | expr
    ;

literal
    : STRING
    | INT
    | DURATION
```

```
    | bool_lit
    | severity_lit
    | list_lit
    ;

bool_lit
    : TRUE
    | FALSE
    ;

severity_lit
    : LOW
    | MEDIUM
    | HIGH
    | CRITICAL
    ;

PLAYBOOK    : 'PLAYBOOK' ;
CONFIG      : 'CONFIG' ;
TRIGGER     : 'TRIGGER' ;
PHASE       : 'PHASE' ;
DO          : 'DO' ;
IF          : 'IF' ;
THEN        : 'THEN' ;
ELSE        : 'ELSE' ;
FOR         : 'FOR' ;
EACH        : 'EACH' ;
IN          : 'IN' ;
WAIT        : 'WAIT' ;
FROM        : 'FROM' ;
TIMEOUT     : 'TIMEOUT' ;
ON_TIMEOUT  : 'ON_TIMEOUT' ;
ON_FAIL     : 'ON_FAIL' ;
VERIFY      : 'VERIFY' ;
LOG         : 'LOG' ;
GOTO        : 'GOTO' ;
```

```
REPORT      : 'REPORT' ;
AND         : 'AND' ;
OR          : 'OR' ;
NOT         : 'NOT' ;
PARALLEL    : 'PARALLEL' ;
SEVERITY    : 'SEVERITY' ;
IMPORT      : 'IMPORT' ;
AS          : 'AS' ;
TRUE        : 'TRUE' ;
FALSE       : 'FALSE' ;
LOW         : 'LOW' ;
MEDIUM      : 'MEDIUM' ;
HIGH        : 'HIGH' ;
CRITICAL    : 'CRITICAL' ;

IDENTIFIER  : [a-zA-Z_][a-zA-Z0-9_]* ;
STRING      : '"' (~["\\\r\n] | '\\' .)* '"' ;
DURATION    : [0-9]+ ('sec' | 'min' | 'hr') ;
INT         : [0-9]+ ;

EQ          : '==' ;
NEQ         : '!=' ;
GTE         : '>=' ;
LTE         : '<=' ;
GT          : '>' ;
LT          : '<' ;

COLON       : ':' ;
COMMA       : ',' ;
DOT         : '.' ;
LPAREN      : '(' ;
RPAREN      : ')' ;
LBRACKET    : '[' ;
RBRACKET    : ']' ;
ASSIGN      : '=' ;
ARROW       : '->' ;
```

```
COMMENT       : '#' ~[\r\n]* -> skip ;
BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
WHITESPACE    : [ \t\r\n]+ -> skip ;
BOM           : '\uFEFF' -> skip ;
```

A.2 Parser Driver Excerpt

```
public class Main {

    public static void main(String[] args) throws IOException {
        if (args.length < 1) {
            System.err.println("Usage: Main <source-file.iflow>");
            System.exit(1);
        }

        String source = new String(Files.readAllBytes(Paths.get(args[0])));

        CharStream input = CharStreams.fromString(source);
        IncidentFlowLexer lexer = new IncidentFlowLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        IncidentFlowParser parser = new IncidentFlowParser(tokens);

        ErrorCounter errors = new ErrorCounter();
        lexer.removeErrorListeners();
        lexer.addErrorListener(errors);
        parser.removeErrorListeners();
        parser.addErrorListener(errors);

        ParseTree tree = parser.program();

        if (errors.getCount() > 0) {
            System.err.println("\nParsing finished with " + errors.getCount() + " error(s
    ).");
            System.exit(2);
        }

```

```
        Future<JFrame> dialog = Trees.inspect(tree, parser);
        try {
            dialog.get();
        } catch (Exception e) {
            System.err.println("Tree viewer error: " + e.getMessage());
        }
    }
}
```

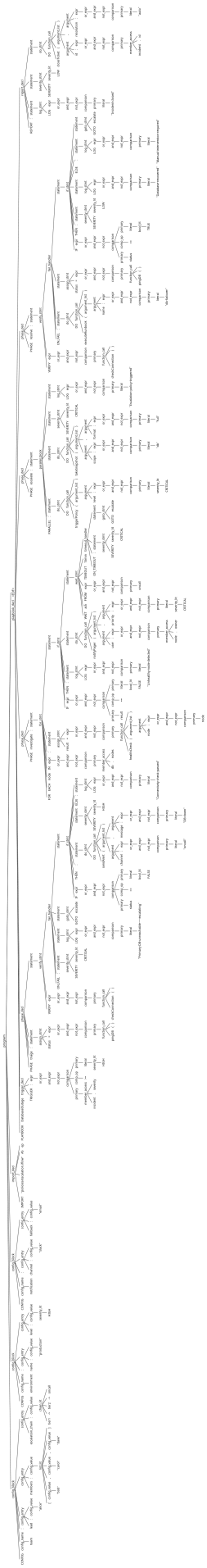
A.3 Error Listener Implementation

```
private static class ErrorCounter extends BaseErrorListener {
    private int count = 0;

    @Override
    public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
                            int line, int charPositionInLine,
                            String msg, RecognitionException e) {
        System.err.printf("line %d:%d %s%n", line, charPositionInLine, msg);
        count++;
    }

    int getCount() { return count; }
}
```

A.4 Derivation Tree Figure



A.5 Playbook Sample

```
CONFIG team:
    lead: "alice"
    members: ["bob", "carol", "dave"]
    escalation_chain: tier1 -> tier2 -> oncall

CONFIG environment:
    name: "production"
    level: HIGH

IMPORT "policies/escalation.iflow" AS ep

PLAYBOOK DatabaseOutage
    TRIGGER: incident.severity >= HIGH

    PHASE triage:
        status = pingDB()
        VERIFY checkConnection()
        ON_FAIL:
            SEVERITY CRITICAL
            LOG "Primary DB unreachable - escalating"
            GOTO escalate

        IF status == FALSE THEN:
            DO sendAlert(channel: "oncall", message: "DB down")
            SEVERITY HIGH
        ELSE:
            LOG "Connectivity check passed"

    PHASE investigate:
        FOR EACH node IN db.nodes:
            result = healthCheck(node: node)
            IF result == FALSE THEN:
                LOG "Unhealthy node detected"
                DO notifyPager(user: node.owner, priority: CRITICAL)
```

```
        WAIT ack FROM oncall TIMEOUT 15min
        ON_TIMEOUT:
            SEVERITY CRITICAL
            GOTO escalate

    PHASE escalate:
        PARALLEL:
            DO triggerPolicy(level: CRITICAL)
            DO takeSnapshot(scope: "db", format: "full")
        SEVERITY CRITICAL
        LOG "Escalation policy triggered"

    REPORT:
        LOG "Incident closed"
        SEVERITY LOW
        DO closeTicket(id: incident.id, resolution: "auto")
```

A.6 Problem Map

a)	Present the problem situation in 5-7 sentences.	When a cybersecurity incident occurs such as a ransomware attack, data breach, or brute force intrusion organizations rely on incident response playbooks to guide their teams through detection, containment, eradication, and recovery. Currently, these playbooks exist as static Word documents, PDFs, or wiki pages that team members must read and follow manually under extreme time pressure. This manual approach leads to skipped steps, miscommunication, delayed escalations, and inconsistent responses. There is no way to test or validate a playbook before a real incident occurs. Existing automation tools SOAR platforms address this through drag-and-drop visual interfaces, but these are vendor-locked, expensive, and cannot be version-controlled, peer-reviewed, or tested like code. Security teams need a way to express their response procedures in a format that is both human-readable and machine-executable, bridging the gap between policy documentation and operational automation
b)	Formulate the problem – 1 statement.	Incident response playbooks are currently static, untestable documents that cannot be validated or executed automatically, leading to slow, inconsistent, and error-prone responses during cybersecurity incidents
TYPE OF QUESTION	**EXAMPLES OF QUESTIONS**	**SHORT ANSWER**
Who…?	1. Who has a problem? 2. Who is involved in the problem situation? 3. Who suffers from the problem? 4. Who should take part in solving the problem?	1. Security Operations Center (SOC) teams, incident responders, and CISOs who are responsible for handling cybersecurity incidents

		2. SOC analysts, incident response leads, forensics specialists, IT administrators, compliance officers, and management who approve and audit response procedures 3. The entire organization because delayed or inconsistent incident response leads to greater financial damage, data loss, regulatory penalties, and reputational harm 4. Security engineers, DSL/language designers, and domain experts in incident response who can define the correct playbook structures and validate the language design
What …? Which…?	5. What happened? 6. What is the problem? 7. What led you to identify the problem (reasons, causes, evidence, arguments, findings, survey results, etc.)? 8. What are the causes that affect those involved? 9. What are the needs of those targeted? 10. What are the resources (financial, human, logistical, time, etc.) necessary to solve the problem?	5. Incident response procedures remain trapped in static documents while every other part of the security pipeline like detection, monitoring, alerting has been automated. 6. There is no dedicated language that allows IR teams to write, validate, and execute incident response playbooks as code. 7. Analysis of existing IR workflows shows that playbooks from organizations like CISA and CERT Société Générale are distributed as PDFs and Word documents. Industry reports indicate that the average breach takes over 100 days to identify and remediate, partly due to manual, unstructured response processes

		8. Lack of automation tools accessible to non-developers, reliance on static documentation, absence of pre-execution validation, vendor lock-in of existing SOAR platforms, and high cost of commercial solutions 9. A simple, readable, domain-specific language that allows writing playbooks in familiar IR terminology, with static validation, automated execution, team configuration, and report generation without requiring programming expertise 10.A team of 4 developers, one semester of development time, Python as the host language, the Lark parser library, and access to publicly available playbook templates for reference and testing
Where…?	11. Where did the problem arise (location, area, stage of the process of the problem arising)?	11. In Security Operations Centers and incident response teams across organizations of all sizes, where the gap between written policy and operational execution remains unaddressed
When…?	12. When did the problem arise? 13. When was the problem identified? 14. When is it planned to be resolved?	The problem has existed since organizations began formalizing incident response procedures, but has become critical as cyberattacks have grown in frequency and sophistication over the past decade During the analysis phase of this project, when reviewing how real-world IR teams operate and comparing their static playbook documents against the automation

		capabilities available in other security domains By the end of the current semester, delivering a functional DSL prototype with parser, validator, interpreter, and example playbooks
Why…?	15. Why is there a need to solve the problem?	15. Because manual incident response is too slow and error-prone for modern cyber threats. Faster, validated, and automated response directly reduces breach impact, financial loss, and recovery time
How...?	16. How (in what way) can the problem be solved? 17. How will the elimination of the problem be determined?	16. By designing and implementing IncidentFlow a domain-specific language that allows IR teams to write playbooks as executable code with phases, triggers, conditionals, escalation chains, and automated report generation 17. By demonstrating that playbooks written in IncidentFlow can be statically validated for correctness, executed against simulated incidents, and produce accurate timeline reports all without requiring the user to know a general-purpose programming language
How much...?	18. How big is the problem? 19. How much is the quantity/quality/number of people etc. affected? 20. How long will it take to resolve the problem? 21. How many parties need to be involved in resolving it?	18. How big is the problem? It affects virtually every organization with a cybersecurity function. Even large enterprises with SOAR tools struggle with playbook maintenance and validation. 19. Thousands of SOC teams globally. According to industry

		data, the cybersecurity workforce gap exceeds 3.5 million people 20.The prototype will be developed over one semester 21.A team of 4 members covering language design, parser implementation, execution engine, validation, testing, and documentation
Other questions you found answered (optional)	22. 23.	-
	How do you see solving this problem (in terms of deliverable/product/result)?	a working DSL with a parser, static validator, simulated execution engine, and report generator, accompanied by three example playbooks and a comprehensive project report documenting the design and implementation
	Conclusions, findings, reflections...	Current incident response practices rely heavily on human interpretation of static documents under high-stress conditions. While SOAR platforms partially address this, they are expensive, vendor-locked, and GUI-based making them impossible to version-control or peer-review

PBL PROBLEM MAP

Bibliography

[1] University of Chicago Press. The Chicago Manual of Style Online. Online, 2023. Available from: https://www.chicagomanualofstyle.org/

[2] IBM Security. Cost of a Data Breach Report 2023. Technical report, IBM Corporation, 2023. Available from: https://www.ibm.com/reports/data-breach

[3] Cichonski, P.; Millar, T.; et al. Computer Security Incident Handling Guide (SP 800-61 Rev. 2). Technical report, National Institute of Standards and Technology, 2012. Available from: https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final

[4] International Organization for Standardization. ISO/IEC 27035 Information security, cybersecurity and privacy protection — Managing information security incidents. 2023.

[5] SANS Institute. Incident Handler's Handbook. White Paper / Online, 2021.

[6] Ponemon Institute. The State of the SOC Report 2022. Technical report, Ponemon Institute, 2022.

[7] ISACA. State of Cybersecurity 2021. Technical report, ISACA, 2021.

[8] Dutton, W. H.; Creese, S.; et al. Human factors in cyber incident response: cognitive load and operational performance. *Journal of Cybersecurity*, volume 6, no. 1, 2020.

[9] Gartner. Market Guide for Security Orchestration, Automation and Response Solutions. Research Report, 2023.

[10] European Parliament and Council of the European Union. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, 2016.

[11] European Parliament and Council of the European Union. Directive (EU) 2022/2555 on measures for a high common level of cybersecurity across the Union (NIS2). Official Journal of the European Union, 2022.

[12] U.S. Congress. Sarbanes-Oxley Act of 2002. Public Law 107–204, 2002.

[13] U.S. Congress. Health Insurance Portability and Accountability Act of 1996. Public Law 104–191, 1996.